

对麻省理工学院的 Dave Clarke 提出的“粗略一致和能用的代码”的投入。

因特网的种子是在 1957 年播下的，其时正值冷战的高峰，苏联发射 Sputnik，第一颗人造地球卫星，震惊了世界。作为响应，美国政府创建了高级研究计划署(ARPA)，其任务就是重建美国在科学与技术上的领导地位。1967 年，ARPA 的 Lawrence Roberts 提出了一个计划，建立一个叫做 ARPANET 的新网络。第一个 ARPANET 节点是在 1969 年建立并运行的。到 1971 年，已有 13 个 ARPANET 节点，而且 email 作为第一个重要的网络应用涌现出来。

1972 年，Robert Kahn 概括了网络互联的一般原则：一组互相连接的网络，通过叫做“路由器”的黑盒子按照“以尽力传送作为基础”在互相独立处理的网络间实现通信。1974 年，Kahn 和 Vinton Cerf 发表了 TCP/IP 协议的第一本详细资料，到 1982 年它成为了 ARPANET 的标准网络互联协议。1983 年 1 月 1 日，ARPANET 的每个节点都切换到 TCP/IP，标志着全球 IP 因特网的诞生。

1985 年，Paul Mockapetris 发明了 DNS，有 1000 多台因特网主机。1986 年，国家科学基金会(NSF)用 56KB/s 的电话线连接了 13 个节点，构建了 NSFNET 的骨干网。其后在 1988 年升级到 1.5MB/s T1 的连接速率，1991 年为 45MB/s T3 的连接速率。到 1988 年，有超过 50 000 台主机。1989 年，原始的 ARPANET 正式退休了。1995 年，已经有几乎 10 000 000 台因特网主机了，NSF 取消了 NSFNET，并且用基于由公众网络接入点连接的私有商业骨干网的现代因特网架构取代了它。

11.4 套接字接口

套接字接口(socket interface)是一组函数，它们和 Unix I/O 函数结合起来，用以创建网络应用。大多数现代系统上都实现套接字接口，包括所有的 Unix 变种、Windows 和 Macintosh 系统。图 11-12 给出了一个典型的客户端-服务器事务的上下文中的套接字接口概述。当讨论各个函数时，你可以使用这张图来作为向导图。

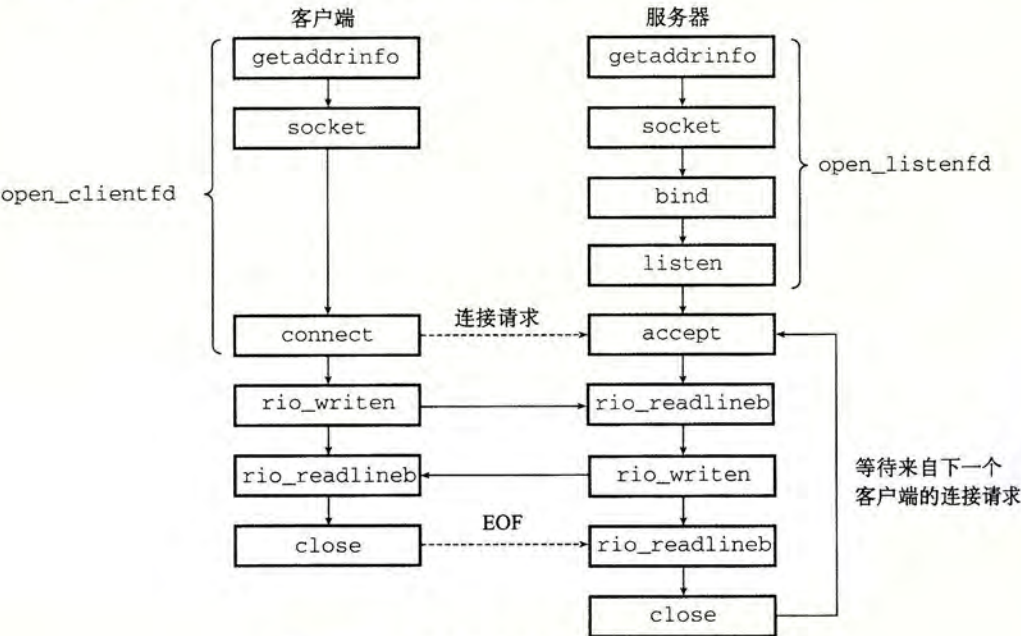


图 11-12 基于套接字接口的网络应用概述

**旁注 套接字接口的起源**

套接字接口是加州大学伯克利分校的研究人员在 20 世纪 80 年代早期提出的。因为这个原因，它也经常被称为伯克利套接字。伯克利研究者使得套接字接口适用于任何底层的协议。第一个实现的就是针对 TCP/IP 协议的，他们把它包括在 Unix 4.2BSD 的内核里，并且分发给许多学校和实验室。这在因特网的历史上是一个重大事件。几乎一夜之间，成千上万的人们接触到了 TCP/IP 和它的源代码。它引起了巨大的轰动，并激发了新的网络和网络互联研究的浪潮。

**11.4.1 套接字地址结构**

从 Linux 内核的角度来看，一个套接字就是通信的一个端点。从 Linux 程序的角度来看，套接字就是一个有相应描述符的打开文件。

因特网的套接字地址存放在如图 11-13 所示的类型为 `sockaddr_in` 的 16 字节结构中。对于因特网应用，`sin_family` 成员是 `AF_INET`，`sin_port` 成员是一个 16 位的端口号，而 `sin_addr` 成员就是一个 32 位的 IP 地址。IP 地址和端口号总是以网络字节顺序（大端法）存放的。

code/netp/netpfragments.c

```
/* IP socket address structure */
struct sockaddr_in {
    uint16_t      sin_family; /* Protocol family (always AF_INET) */
    uint16_t      sin_port;   /* Port number in network byte order */
    struct in_addr sin_addr;   /* IP address in network byte order */
    unsigned char sin_zero[8]; /* Pad to sizeof(struct sockaddr) */
};

/* Generic socket address structure (for connect, bind, and accept) */
struct sockaddr {
    uint16_t sa_family; /* Protocol family */
    char      sa_data[14]; /* Address data */
};
```

code/netp/netpfragments.c

图 11-13 套接字地址结构

**旁注 \_in 后缀意味着什么？**

\_in 后缀是互联网络(internet)的缩写，而不是输入(input)的缩写。

`connect`、`bind` 和 `accept` 函数要求一个指向与协议相关的套接字地址结构的指针。套接字接口的设计者面临的问题是，如何定义这些函数，使之能接受各种类型的套接字地址结构。今天我们可以使用通用的 `void*` 指针，但是那时在 C 中并不存在这种类型的指针。解决办法是定义套接字函数要求一个指向通用 `sockaddr` 结构(图 11-13)的指针，然后要求应用程序将与协议特定的结构的指针强制转换成这个通用结构。为了简化代码示例，我们跟随 Steven 的指导，定义下面的类型：

```
typedef struct sockaddr SA;
```

然后无论何时需要将 `sockaddr_in` 结构强制转换成通用 `sockaddr` 结构时，我们都使用这个类型。

### 11.4.2 socket 函数

客户端和服务端使用 socket 函数来创建一个套接字描述符(socket descriptor)。

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

返回：若成功则为非负描述符，若出错则为-1。

如果想要使套接字成为连接的一个端点，就用如下硬编码的参数来调用 socket 函数：

```
clientfd = Socket(AF_INET, SOCK_STREAM, 0);
```

其中，AF\_INET 表明我们正在使用 32 位 IP 地址，而 SOCK\_STREAM 表示这个套接字是连接的一个端点。不过最好的方法是用 getaddrinfo 函数(11.4.7 节)来自动生成这些参数，这样代码就与协议无关了。我们会在 11.4.8 节中向你展示如何配合 socket 函数来使用 getaddrinfo。

socket 返回的 clientfd 描述符仅是部分打开的，还不能用于读写。如何完成打开套接字的工作，取决于我们是客户端还是服务器。下一节描述当我们是客户端时如何完成打开套接字的工作。

### 11.4.3 connect 函数

客户端通过调用 connect 函数来建立和服务器的连接。

```
#include <sys/socket.h>

int connect(int clientfd, const struct sockaddr *addr,
            socklen_t addrlen);
```

返回：若成功则为 0，若出错则为-1。

connect 函数试图与套接字地址为 addr 的服务器建立一个因特网连接，其中 addrlen 是 sizeof(sockaddr\_in)。connect 函数会阻塞，一直到连接成功建立或是发生错误。如果成功，clientfd 描述符现在就准备好可以读写了，并且得到的连接是由套接字对

```
(x:y, addr.sin_addr:addr.sin_port)
```

刻画的，其中 x 表示客户端的 IP 地址，而 y 表示临时端口，它唯一地确定了客户端主机上的客户端进程。对于 socket，最好的方法是用 getaddrinfo 来为 connect 提供参数(见 11.4.8 节)。

### 11.4.4 bind 函数

剩下的套接字函数——bind、listen 和 accept，服务器用它们来和客户端建立连接。

```
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *addr,
         socklen_t addrlen);
```

返回：若成功则为 0，若出错则为-1。

bind 函数告诉内核将 addr 中的服务器套接字地址和套接字描述符 sockfd 联系起来。参数 addrlen 就是 sizeof(sockaddr\_in)。对于 socket 和 connect, 最好的方法是用 getaddrinfo 来为 bind 提供参数(见 11.4.8 节)。

#### 11.4.5 listen 函数

客户端是发起连接请求的主动实体。服务器是等待来自客户端的连接请求的被动实体。默认情况下, 内核会认为 socket 函数创建的描述符对应于主动套接字(active socket), 它存在于一个连接的客户端。服务器调用 listen 函数告诉内核, 描述符是被服务器而不是客户端使用的。

```
#include <sys/socket.h>

int listen(int sockfd, int backlog);
```

返回: 若成功则为 0, 若出错则为 -1。

listen 函数将 sockfd 从一个主动套接字转化为一个监听套接字(listening socket), 该套接字可以接受来自客户端的连接请求。backlog 参数暗示了内核在开始拒绝连接请求之前, 队列中要排队的未完成的连接请求的数量。backlog 参数的确切含义要求对 TCP/IP 协议的理解, 这超出了我们讨论的范围。通常我们会把它设置为一个较大的值, 比如 1024。

#### 11.4.6 accept 函数

服务器通过调用 accept 函数来等待来自客户端的连接请求。

```
#include <sys/socket.h>

int accept(int listenfd, struct sockaddr *addr, int *addrlen);
```

返回: 若成功则为非负连接描述符, 若出错则为 -1。

accept 函数等待来自客户端的连接请求到达侦听描述符 listenfd, 然后在 addr 中填写客户端的套接字地址, 并返回一个已连接描述符(connected descriptor), 这个描述符可被用来利用 Unix I/O 函数与客户端通信。

监听描述符和已连接描述符之间的区别使很多人感到迷惑。监听描述符是作为客户端连接请求的一个端点。它通常被创建一次, 并存在于服务器的整个生命周期。已连接描述符是客户端和服务器之间已经建立起来了的连接的一个端点。服务器每次接受连接请求时都会创建一次, 它只存在于服务器为一个客户端服务的过程中。

图 11-14 描绘了监听描述符和已连接描述符的角色。在第一步中, 服务器调用 accept, 等待连接请求到达监听描述符, 具体地我们设定为描述符 3。回忆一下, 描述符 0~2 是预留给了标准文件的。

在第二步中, 客户端调用 connect 函数, 发送一个连接请求到 listenfd。第三步, accept 函数打开了一个新的已连接描述符 connfd(我们假设是描述符 4), 在 clientfd 和 connfd 之间建立连接, 并且随后返回 connfd 给应用程序。客户端也从 connect 返回, 在这一点以后, 客户端和服务器就可以分别通过读和写 clientfd 和 connfd 来回传送数据了。

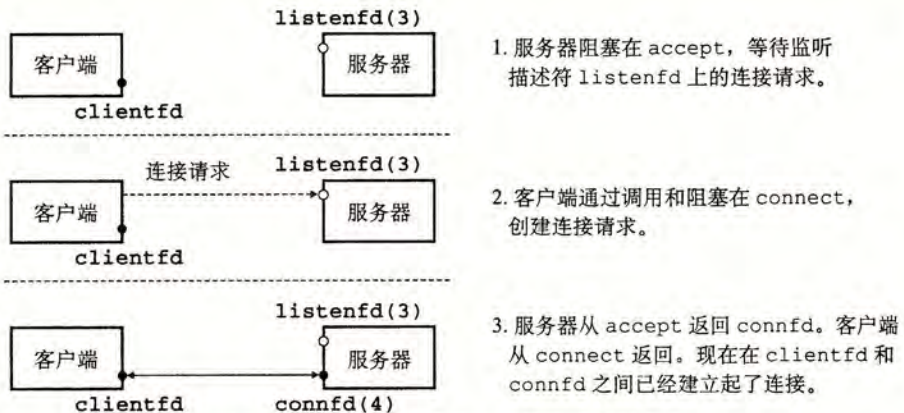


图 11-14 监听描述符和已连接描述符的角色

**旁注** 为何要有监听描述符和已连接描述符之间的区别？

你可能很想知道为什么套接字接口要区别监听描述符和已连接描述符。乍一看，这像是不必要的复杂化。然而，区分这两者被证明是很有用的，因为它使得我们可以建立并发服务器，它能够同时处理许多客户端连接。例如，每次一个连接请求到达监听描述符时，我们可以派生(fork)一个新的进程，它通过已连接描述符与客户端通信。在第 12 章中将介绍更多关于并发服务器的内容。

### 11.4.7 主机和服务的转换

Linux 提供了一些强大的函数(称为 `getaddrinfo` 和 `getnameinfo`)实现二进制套接字地址结构和主机名、主机地址、服务名和端口号的字符串表示之间的相互转化。当和套接字接口一起使用时，这些函数能使我们编写独立于任何特定版本的 IP 协议的网络程序。

#### 1. `getaddrinfo` 函数

`getaddrinfo` 函数将主机名、主机地址、服务名和端口号的字符串表示转化成套接字地址结构。它是已弃用的 `gethostbyname` 和 `getservbyname` 函数的新的替代品。和以前的那些函数不同，这个函数是可重入的(见 12.7.2 节)，适用于任何协议。

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *host, const char *service,
                const struct addrinfo *hints,
                struct addrinfo **result);
                返回：如果成功则为 0，如果错误则为非零的错误代码。

void freeaddrinfo(struct addrinfo *result);
                返回：无。

const char *gai_strerror(int errcode);
                返回：错误消息。
```

给定 `host` 和 `service`(套接字地址的两个组成部分)，`getaddrinfo` 返回 `result`，`result` 一个指向 `addrinfo` 结构的链表，其中每个结构指向一个对应于 `host` 和 `service` 的套接字地址结构(图 11-15)。

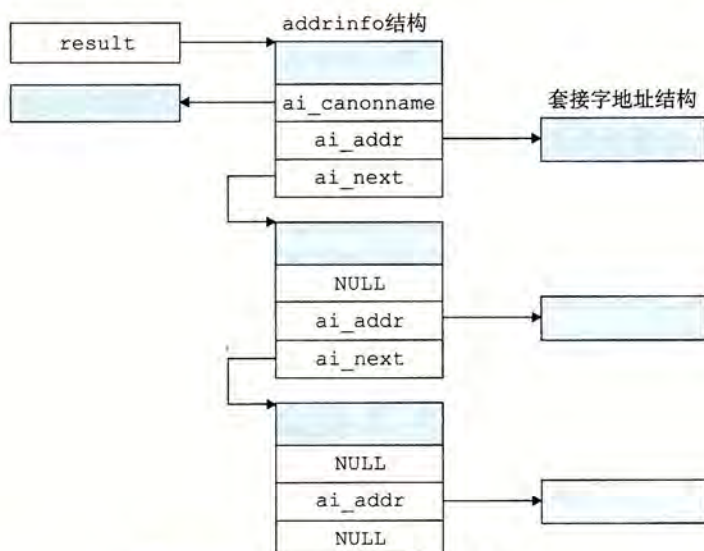


图 11-15 getaddrinfo 返回的数据结构

在客户端调用了 `getaddrinfo` 之后，会遍历这个列表，依次尝试每个套接字地址，直到调用 `socket` 和 `connect` 成功，建立起连接。类似地，服务器会尝试遍历列表中的每个套接字地址，直到调用 `socket` 和 `bind` 成功，描述符会被绑定到一个合法的套接字地址。为了避免内存泄漏，应用程序必须在最后调用 `freeaddrinfo`，释放该链表。如果 `getaddrinfo` 返回非零的错误代码，应用程序可以调用 `gai_strerror`，将该代码转换成消息字符串。

`getaddrinfo` 的 `host` 参数可以是域名，也可以是数字地址（如点分十进制 IP 地址）。`service` 参数可以是服务名（如 `http`），也可以是十进制端口号。如果不想把主机名转换成地址，可以把 `host` 设置为 `NULL`。对 `service` 来说也是一样。但是必须指定两者中至少一个。

可选的参数 `hints` 是一个 `addrinfo` 结构（见图 11-16），它提供对 `getaddrinfo` 返回的套接字地址列表的更好的控制。如果要传递 `hints` 参数，只能设置下列字段：`ai_family`、`ai_socktype`、`ai_protocol` 和 `ai_flags` 字段。其他字段必须设置为 0（或 `NULL`）。实际中，我们用 `memset` 将整个结构清零，然后有选择地设置一些字段：

- `getaddrinfo` 默认可以返回 IPv4 和 IPv6 套接字地址。`ai_family` 设置为 `AF_INET` 会将列表限制为 IPv4 地址；设置为 `AF_INET6` 则限制为 IPv6 地址。
- 对于 `host` 关联的每个地址，`getaddrinfo` 函数默认最多返回三个 `addrinfo` 结构，每个的 `ai_socktype` 字段不同：一个是连接，一个是数据报（本书未讲述），一个是原始套接字（本书未讲述）。`ai_socktype` 设置为 `SOCK_STREAM` 将列表限制为对每个地址最多一个 `addrinfo` 结构，该结构的套接字地址可以作为连接的一个端点。这是所有示例程序所期望的行为。
- `ai_flags` 字段是一个位掩码，可以进一步修改默认行为。可以把各种值用 OR 组合起来得到该掩码。下面是一些我们认为有用的值：

**AI\_ADDRCONFIG**。如果在使用连接，就推荐使用这个标志 [34]。它要求只有当本地主机被配置为 IPv4 时，`getaddrinfo` 返回 IPv4 地址。对 IPv6 也是类似。

**AI\_CANONNAME**。`ai_canonname` 字段默认为 `NULL`。如果设置了该标志，就是告诉 `getaddrinfo` 将列表中第一个 `addrinfo` 结构的 `ai_canonname` 字段指向 `host` 的权威（官方）名字（见图 11-15）。

AI\_NUMERICSERV。参数 service 默认可以是服务名或端口号。这个标志强制参数 service 为端口号。

AI\_PASSIVE。getaddrinfo 默认返回套接字地址，客户端可以在调用 connect 时用作主动套接字。这个标志告诉该函数，返回的套接字地址可能被服务器用作监听套接字。在这种情况下，参数 host 应该为 NULL。得到的套接字地址结构中的地址字段会是通配符地址(wildcard address)，告诉内核这个服务器会接受发送到该主机所有 IP 地址的请求。这是所有示例服务器所期望的行为。

---

```

code/netp/netpfragments.c
struct addrinfo {
    int          ai_flags;      /* Hints argument flags */
    int          ai_family;     /* First arg to socket function */
    int          ai_socktype;   /* Second arg to socket function */
    int          ai_protocol;   /* Third arg to socket function */
    char         *ai_canonname; /* Canonical hostname */
    size_t       ai_addrlen;    /* Size of ai_addr struct */
    struct sockaddr *ai_addr;    /* Ptr to socket address structure */
    struct addrinfo *ai_next;   /* Ptr to next item in linked list */
};

```

---

code/netp/netpfragments.c

图 11-16 getaddrinfo 使用的 addrinfo 结构

当 getaddrinfo 创建输出列表中的 addrinfo 结构时，会填写每个字段，除了 ai\_flags。ai\_addr 字段指向一个套接字地址结构，ai\_addrlen 字段给出这个套接字地址结构的大小，而 ai\_next 字段指向列表中下一个 addrinfo 结构。其他字段描述这个套接字地址的各种属性。

getaddrinfo 一个很好的方面是 addrinfo 结构中的字段是不透明的，即它们可以直接传递给套接字接口中的函数，应用程序代码无需再做任何处理。例如，ai\_family、ai\_socktype 和 ai\_protocol 可以直接传递给 socket。类似地，ai\_addr 和 ai\_addrlen 可以直接传递给 connect 和 bind。这个强大的属性使得我们编写的客户端和服务端能够独立于某个特殊版本的 IP 协议。

## 2. getnameinfo 函数

getnameinfo 函数和 getaddrinfo 是相反的，将一个套接字地址结构转换成相应的主机和服务名字符串。它是已弃用的 gethostbyaddr 和 getservbyport 函数的新的替代品，和以前的那些函数不同，它是可重入和与协议无关的。

```

#include <sys/socket.h>
#include <netdb.h>

int getnameinfo(const struct sockaddr *sa, socklen_t salen,
                char *host, size_t hostlen,
                char *service, size_t servlen, int flags);

```

返回：如果成功则为 0，如果错误则为非零的错误代码。

参数 sa 指向大小为 salen 字节的套接字地址结构，host 指向大小为 hostlen 字节的缓冲区，service 指向大小为 servlen 字节的缓冲区。getnameinfo 函数将套接字地址结构 sa 转换成对应的主机和服务名字符串，并将它们复制到 host 和 service 缓冲区。如果 getnam-

errno 返回非零的错误代码，应用程序可以调用 gai\_strerror 把它转化成字符串。

如果不要主机名，可以把 host 设置为 NULL，hostlen 设置为 0。对服务字段来说也是一样。不过，两者必须设置其中之一。

参数 flags 是一个位掩码，能够修改默认的行为。可以把各种值用 OR 组合起来得到该掩码。下面是两个有用的值：

- NI\_NUMERICHOST。getnameinfo 默认试图返回 host 中的域名。设置该标志会使该函数返回一个数字地址字符串。
- NI\_NUMERICSERV。getnameinfo 默认会检查/etc/services，如果可能，会返回服务名而不是端口号。设置该标志会使该函数跳过查找，简单地返回端口号。

图 11-17 给出了一个简单的程序，称为 HOSTINFO，它使用 getaddrinfo 和 getnameinfo 展示出域名到和它相关联的 IP 地址之间的映射。该程序类似于 11.3.2 节中的 NSLOOKUP 程序。

```

1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct addrinfo *p, *listp, hints;
6      char buf[MAXLINE];
7      int rc, flags;
8
9      if (argc != 2) {
10         fprintf(stderr, "usage: %s <domain name>\n", argv[0]);
11         exit(0);
12     }
13
14     /* Get a list of addrinfo records */
15     memset(&hints, 0, sizeof(struct addrinfo));
16     hints.ai_family = AF_INET; /* IPv4 only */
17     hints.ai_socktype = SOCK_STREAM; /* Connections only */
18     if ((rc = getaddrinfo(argv[1], NULL, &hints, &listp)) != 0) {
19         fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(rc));
20         exit(1);
21     }
22
23     /* Walk the list and display each IP address */
24     flags = NI_NUMERICHOST; /* Display address string instead of domain name */
25     for (p = listp; p; p = p->ai_next) {
26         Getnameinfo(p->ai_addr, p->ai_addrlen, buf, MAXLINE, NULL, 0, flags);
27         printf("%s\n", buf);
28     }
29
30     /* Clean up */
31     Freeaddrinfo(listp);
32
33     exit(0);
34 }

```

code/netp/hostinfo.c

code/netp/hostinfo.c

图 11-17 HOSTINFO 展示出域名到和它相关联的 IP 地址之间的映射

首先，初始化 hints 结构，使 getaddrinfo 返回我们想要的地址。在这里，我们想查找 32 位的 IP 地址(第 16 行)，用作连接的端点(第 17 行)。因为只想 getaddrinfo 转换域名，所以用 service 参数为 NULL 来调用它。

调用 getaddrinfo 之后，会遍历 addrinfo 结构，用 getnameinfo 将每个套接字地址转换成点分十进制地址字符串。遍历完列表之后，我们调用 freeaddrinfo 小心地释放这个列表(虽然对于这个简单的程序来说，并不是严格需要这样做的)。

运行 HOSTINFO 时，我们看到 twitter.com 映射到了四个 IP 地址，和 11.3.2 节用 NSLOOKUP 的结果一样。

```
linux> ./hostinfo twitter.com
199.16.156.102
199.16.156.230
199.16.156.6
199.16.156.70
```

 **练习题 11.4** 函数 getaddrinfo 和 getnameinfo 分别包含了 inet\_pton 和 inet\_ntop 的功能，提供了更高级别的、独立于任何特殊地址格式的抽象。想看看这到底有多方便，编写 HOSTINFO(图 11-17)的一个版本，用 inet\_pton 而不是 getnameinfo 将每个套接字地址转换成点分十进制地址字符串。

#### 11.4.8 套接字接口的辅助函数

初学时，getnameinfo 函数和套接字接口看上去有些可怕。用高级的辅助函数包装一下会方便很多，称为 open\_clientfd 和 open\_listenfd，客户端和服务端互相通信时可以使用这些函数。

##### 1. open\_clientfd 函数

客户端调用 open\_clientfd 建立与服务器的连接。

```
#include "csapp.h"

int open_clientfd(char *hostname, char *port);
```

返回：若成功则为描述符，若出错则为 -1。

open\_clientfd 函数建立与服务器的连接，该服务器运行在主机 hostname 上，并在端口号 port 上监听连接请求。它返回一个打开的套接字描述符，该描述符准备好了，可以用 Unix I/O 函数做输入和输出。图 11-18 给出了 open\_clientfd 的代码。

我们调用 getaddrinfo，它返回 addrinfo 结构的列表，每个结构指向一个套接字地址结构，可用于建立与服务器的连接，该服务器运行在 hostname 上并监听 port 端口。然后遍历该列表，依次尝试列表中的每个条目，直到调用 socket 和 connect 成功。如果 connect 失败，在尝试下一个条目之前，要小心地关闭套接字描述符。如果 connect 成功，我们会释放列表内存，并把套接字描述符返回给客户端，客户端可以立即开始用 Unix I/O 与服务器通信了。

注意，所有的代码都与任何版本的 IP 无关。socket 和 connect 的参数都是用 getaddrinfo 自动产生的，这使得我们的代码干净可移植。

##### 2. open\_listenfd 函数

调用 open\_listenfd 函数，服务器创建一个监听描述符，准备好接收连接请求。

```
#include "csapp.h"
```

```
int open_listenfd(char *port);
```

返回：若成功则为描述符，若出错则为 -1。

code/src/csapp.c

```
1 int open_clientfd(char *hostname, char *port) {
2     int clientfd;
3     struct addrinfo hints, *listp, *p;
4
5     /* Get a list of potential server addresses */
6     memset(&hints, 0, sizeof(struct addrinfo));
7     hints.ai_socktype = SOCK_STREAM; /* Open a connection */
8     hints.ai_flags = AI_NUMERICSERV; /* ... using a numeric port arg. */
9     hints.ai_flags |= AI_ADDRCONFIG; /* Recommended for connections */
10    Getaddrinfo(hostname, port, &hints, &listp);
11
12    /* Walk the list for one that we can successfully connect to */
13    for (p = listp; p; p = p->ai_next) {
14        /* Create a socket descriptor */
15        if ((clientfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol))
16            < 0) continue; /* Socket failed, try the next */
17
18        /* Connect to the server */
19        if (connect(clientfd, p->ai_addr, p->ai_addrlen) != -1)
20            break; /* Success */
21        Close(clientfd); /* Connect failed, try another */
22    }
23
24    /* Clean up */
25    Freeaddrinfo(listp);
26    if (!p) /* All connects failed */
27        return -1;
28    else /* The last connect succeeded */
29        return clientfd;
30 }
```

code/src/csapp.c

图 11-18 open\_clientfd：和服务器建立连接的辅助函数。它是可重入和与协议无关的

open\_listenfd 函数打开和返回一个监听描述符，这个描述符准备好在端口 port 上接收连接请求。图 11-19 展示了 open\_listenfd 的代码。

open\_listenfd 的风格类似于 open\_clientfd。调用 getaddrinfo，然后遍历结果列表，直到调用 socket 和 bind 成功。注意，在第 20 行，我们使用 setsockopt 函数（本书中没有讲述）来配置服务器，使得服务器能够被终止、重启和立即开始接收连接请求。一个重启的服务器默认将在大约 30 秒内拒绝客户端的连接请求，这严重地阻碍了调试。

因为我们调用 getaddrinfo 时，使用了 AI\_PASSIVE 标志并将 host 参数设置为 NULL，每个套接字地址结构中的地址字段会被设置为通配符地址，这告诉内核这个服务器会接收发送到本主机所有 IP 地址的请求。

code/src/csapp.c

```

1  int open_listenfd(char *port)
2  {
3      struct addrinfo hints, *listp, *p;
4      int listenfd, optval=1;
5
6      /* Get a list of potential server addresses */
7      memset(&hints, 0, sizeof(struct addrinfo));
8      hints.ai_socktype = SOCK_STREAM;          /* Accept connections */
9      hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG; /* ... on any IP address */
10     hints.ai_flags |= AI_NUMERICSERV;          /* ... using port number */
11     Getaddrinfo(NULL, port, &hints, &listp);
12
13     /* Walk the list for one that we can bind to */
14     for (p = listp; p; p = p->ai_next) {
15         /* Create a socket descriptor */
16         if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol))
17             < 0) continue; /* Socket failed, try the next */
18
19         /* Eliminates "Address already in use" error from bind */
20         Setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
21                    (const void *)&optval , sizeof(int));
22
23         /* Bind the descriptor to the address */
24         if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)
25             break; /* Success */
26         Close(listenfd); /* Bind failed, try the next */
27     }
28
29     /* Clean up */
30     Freeaddrinfo(listp);
31     if (!p) /* No address worked */
32         return -1;
33
34     /* Make it a listening socket ready to accept connection requests */
35     if (listen(listenfd, LISTENQ) < 0) {
36         Close(listenfd);
37         return -1;
38     }
39     return listenfd;
40 }

```

code/src/csapp.c

图 11-19 open\_listenfd: 打开并返回监听描述符的辅助函数。它是可重入和与协议无关的

最后, 我们调用 listen 函数, 将 listenfd 转换为一个监听描述符, 并返回给调用者。如果 listen 失败, 我们要小心地避免内存泄漏, 在返回前关闭描述符。

#### 11.4.9 echo 客户端和服务器的示例

学习套接字接口的最好方法是研究示例代码。图 11-20 展示了一个 echo 客户端的代

码。在和服务器建立连接之后，客户端进入一个循环，反复从标准输入读取文本行，发送文本行给服务器，从服务器读取回送的行，并输出结果到标准输出。当 `fgets` 在标准输入上遇到 EOF 时，或者因为用户在键盘上键入 Ctrl+D，或者因为在一个重定向的输入文件中用尽了所有的文本行时，循环就终止。

---

```
code/netp/echoclient.c
```

```

1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      int clientfd;
6      char *host, *port, buf[MAXLINE];
7      rio_t rio;
8
9      if (argc != 3) {
10         fprintf(stderr, "usage: %s <host> <port>\n", argv[0]);
11         exit(0);
12     }
13     host = argv[1];
14     port = argv[2];
15
16     clientfd = Open_clientfd(host, port);
17     Rio_readinitb(&rio, clientfd);
18
19     while (Fgets(buf, MAXLINE, stdin) != NULL) {
20         Rio_writen(clientfd, buf, strlen(buf));
21         Rio_readlineb(&rio, buf, MAXLINE);
22         Fputs(buf, stdout);
23     }
24     Close(clientfd);
25     exit(0);
26 }

```

---

```
code/netp/echoclient.c
```

图 11-20 echo 客户端的主程序

循环终止之后，客户端关闭描述符。这会导致发送一个 EOF 通知到服务器，当服务器从它的 `reio_readlineb` 函数收到一个为零的返回码时，就会检测到这个结果。在关闭它的描述符后，客户端就终止了。既然客户端内核在一个进程终止时会自动关闭所有打开的描述符，第 24 行的 `close` 就没有必要了。不过，显式地关闭已经打开的任何描述符是一个良好的编程习惯。

图 11-21 展示了 echo 服务器的主程序。在打开监听描述符后，它进入一个无限循环。每次循环都等待一个来自客户端的连接请求，输出已连接客户端的域名和 IP 地址，并调用 `echo` 函数为这些客户端服务。在 `echo` 程序返回后，主程序关闭已连接描述符。一旦客户端和服务端关闭了它们各自的描述符，连接也就终止了。

第 9 行的 `clientaddr` 变量是一个套接字地址结构，被传递给 `accept`。在 `accept` 返回之前，会在 `clientaddr` 中填上连接另一端客户端的套接字地址。注意，我们将 `clientaddr` 声明为 `struct sockaddr_storage` 类型，而不是 `struct sockaddr_in` 类型。根据定义，`sockaddr_storage` 结构足够大能够装下任何类型的套接字地址，以保持代码的协议无关性。

code/netp/echoserveri.c

```

1  #include "csapp.h"
2
3  void echo(int connfd);
4
5  int main(int argc, char **argv)
6  {
7      int listenfd, connfd;
8      socklen_t clientlen;
9      struct sockaddr_storage clientaddr; /* Enough space for any address */
10     char client_hostname[MAXLINE], client_port[MAXLINE];
11
12     if (argc != 2) {
13         fprintf(stderr, "usage: %s <port>\n", argv[0]);
14         exit(0);
15     }
16
17     listenfd = Open_listenfd(argv[1]);
18     while (1) {
19         clientlen = sizeof(struct sockaddr_storage);
20         connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
21         Getnameinfo((SA *) &clientaddr, clientlen, client_hostname, MAXLINE,
22                     client_port, MAXLINE, 0);
23         printf("Connected to (%s, %s)\n", client_hostname, client_port);
24         echo(connfd);
25         Close(connfd);
26     }
27     exit(0);
28 }

```

code/netp/echoserveri.c

图 11-21 迭代 echo 服务器的主程序

注意，简单的 echo 服务器一次只能处理一个客户端。这种类型的服务器一次一个地在客户端间迭代，称为迭代服务器(iterative server)。在第 12 章中，我们将学习如何建立更加复杂的并发服务器(concurrent server)，它能够同时处理多个客户端。

最后，图 11-22 展示了 echo 程序的代码，该程序反复读写文本行，直到 `rio_readlineb` 函数在第 10 行遇到 EOF。

code/netp/echo.c

```

1  #include "csapp.h"
2
3  void echo(int connfd)
4  {
5      size_t n;
6      char buf[MAXLINE];
7      rio_t rio;
8
9      Rio_readinitb(&rio, connfd);
10     while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
11         printf("server received %d bytes\n", (int)n);
12         Rio_writen(connfd, buf, n);
13     }
14 }

```

code/netp/echo.c

图 11-22 读和回送文本行的 echo 函数

**旁注** 在连接中 EOF 意味什么?

EOF 的概念常常使人们感到迷惑,尤其是在因特网连接的上下文中。首先,我们需要理解其实并没有像 EOF 字符这样的一个东西。进一步来说,EOF 是由内核检测到的一种条件。应用程序在它接收到一个由 read 函数返回的零返回码时,它就会发现出 EOF 条件。对于磁盘文件,当前文件位置超出文件长度时,会发生 EOF。对于因特网连接,当一个进程关闭连接它的那一端时,会发生 EOF。连接另一端的进程在试图读取流中最后一个字节之后的字节时,会检测到 EOF。

## 11.5 Web 服务器

迄今为止,我们已经在简单的 echo 服务器的上下文中讨论了网络编程。在这一节里,我们将向你展示如何利用网络编程的基本概念,来创建你自己的虽小但功能齐全的 Web 服务器。

### 11.5.1 Web 基础

Web 客户端和服务端之间的交互用的是一个基于文本的应用级协议,叫做 HTTP (Hypertext Transfer Protocol, 超文本传输协议)。HTTP 是一个简单的协议。一个 Web 客户端(即浏览器)打开一个到服务器的因特网连接,并且请求某些内容。服务器响应所请求的内容,然后关闭连接。浏览器读取这些内容,并把它显示在屏幕上。

Web 服务和常规的文件检索服务(例如 FTP)有什么区别呢?主要的区别是 Web 内容可以用一种叫做 HTML(Hypertext Markup Language, 超文本标记语言)的语言来编写。一个 HTML 程序(页)包含指令(标记),它们告诉浏览器如何显示这页中的各种文本和图形对象。例如,代码

```
<b> Make me bold! </b>
```

告诉浏览器用粗体字类型输出 `<b>` 和 `</b>` 标记之间的文本。然而,HTML 真正的强大之处在于一个页面可以包含指针(超链接),这些指针可以指向存放在任何因特网主机上的内容。例如,一个格式如下的 HTML 行

```
<a href="http://www.cmu.edu/index.html">Carnegie Mellon</a>
```

告诉浏览器高亮显示文本对象“Carnegie Mellon”,并且创建一个超链接,它指向存放在 CMU Web 服务器上叫做 index.html 的 HTML 文件。如果用户单击了这个高亮文本对象,浏览器就会从 CMU 服务器中请求相应的 HTML 文件并显示它。

**旁注** 万维网的起源

万维网是 Tim Berners-Lee 发明的,他是一位在瑞典物理实验室 CERN(欧洲粒子物理研究所)工作的软件工程师。1989 年, Berners-Lee 写了一个内部备忘录,提出了一个分布式超文本系统,它能连接“用链接组成的笔记的网(web of notes with links)”。提出这个系统的目的是帮助 CERN 的科学家共享和管理信息。在接下来的两年多里, Berners-Lee 实现了第一个 Web 服务器和 Web 浏览器之后,在 CERN 内部以及其他一些网站中, Web 发展出了小规模拥护者。1993 年一个关键事件发生了, Marc Andreessen(他后来创建了 Netscape)和他在 NCSA 的同事发布了一种图形化的浏览器,叫做 MOSAIC,可以在三种主要的平台上所使用: Unix、Windows 和 Macintosh。在 MOSAIC 发布后,对 Web 的兴趣爆发了, Web 网站以每年 10 倍或更高的数量增长。到 2015 年,世界上已经有超过 975 000 000 个 Web 网站了(源自 Netcraft Web Survey)。